

Tabăra de pregătire a lotului național de informatică juniori

Bistrița, 10-15 mai 2026

Descrierea soluțiilor Baraj 4

Comisia științifică

14 mai 2026

Problema 1. Hanoi

Propusă de: stud. Aureliu Valentin Antonie, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București

Subtask $K = 2$, diametrele discurilor sunt distincte

Pornim de la observația asupra faptului că dacă selectăm o submulțime de discuri pe care să o plasăm pe o tijă, diametrele fiind distincte, le vom putea plasa de fiecare dată strict crescător, în mod unic, pe tijă respectivă. Fie D = mulțimea diametrelor discurilor. K fiind egal cu 2, putem să construim soluția plasând o submulțime $A \subseteq D$ pe tijă 1, iar restul de discuri $D \setminus A$ le vom plasa pe tijă 2. Astfel, răspunsul va fi: $(2^N - 2) \bmod 99\,041$, numărul de submulțimi, eliminând $A = \emptyset$ (tijă 1 nu poate fi liberă) și submulțimea $A = D$, deoarece $D \setminus A = \emptyset$ (tijă 2 nu poate fi liberă) $\bmod 99\,041$.

Complexitate timp: $O(N)$.

Subtask $K = N$, diametrele discurilor sunt distincte

Trebuie să așezăm cele N discuri pe $K = N$ tije, astfel încât nicio tijă să rămână goală. Prin urmare, fiecare tijă trebuie să aibă un singur disc. Numărul de moduri de așezare va fi: $N! \bmod 99\,041$, reprezentând numărul de permutări ale discurilor.

Complexitate timp: $O(N)$.

Subtask Diametrele discurilor sunt distincte, fără alte restricții

Revenind asupra observației de mai sus: faptul că dacă alegem o submulțime, diametrele fiind distincte, o putem așeza strict crescător în mod unic pe o tijă, problema reducându-se la a calcula numerele lui Stirling de speța a II-a. Definim $S(N, K)$ = numărul de partiționări ale unei mulțimi de N elemente în K submulțimi nevide. Dinamica va avea următoarea recurență:

$$S(N, K) = S(N - 1, K - 1) + K \cdot S(N - 1, K)$$

, în care ori începem o nouă submulțime cu elementul curent, ori îl adăugăm la submulțimile precedente. Cazul de bază este $S(N, 1) = 1$. Deoarece ordinea tijelor este importantă, răspunsul final va fi: $(S(N, K) \cdot K!) \bmod 99\,041$ ($K!$ deoarece tijele pot fi permutate).

Complexitate timp: $O(N \cdot K)$.

Notăm: fr_i = frecvența de apariție a unui disc de diametru i . Fie $\{x_1, x_2, \dots, x_p\}$ mulțimea diametrelor distincte.

Subtask $K = 2$, diametrele nu sunt distincte

Observăm faptul că dacă există i astfel încât $fr_i > 2$, nu avem soluție. Altfel, frecvențele vor putea fi: $\{0, 1, 2\}$. Discurile care au diametru i și $fr_i = 2$, trebuie obligatoriu puse pe tije diferite, în mod unic. În concluzie, răspunsul va fi egal cu: $2^{nr} \bmod 99\,041$, unde nr reprezintă numărul discurilor cu diametrul i și $fr_i = 1$. Dacă avem soluție, atunci cu siguranță vom avea frecvențe egale cu 2. Datorită acestui lucru, înseamnă că nu mai trebuie să luăm în considerare cazul de mulțime vidă, deoarece tijele nu au cum să rămână libere.

Complexitate timp: $O(N)$.

Subtask $K = N$, diametrele nu sunt distincte

Pornind de la ideea de la subtaskul 2, putem extinde și la cazul general, folosindu-ne de formula de la permutări cu repetiție. Numărul de așezări va fi: $\left(\frac{N!}{fr_{x_1}! \cdot fr_{x_2}! \cdot \dots \cdot fr_{x_p}!} \right) \bmod 99\,041$, unde $fr_{x_1} + fr_{x_2} + \dots + fr_{x_p} = N$. Pentru a efectua corect împărțirea dată, ne vom folosi de proprietatea inversului modular: $\left(\frac{N!}{fr_{x_1}! \cdot fr_{x_2}! \cdot \dots \cdot fr_{x_p}!} \right) \bmod 99\,041 = (N! \cdot (fr_{x_1}! \cdot fr_{x_2}! \cdot \dots \cdot fr_{x_p}!)^{-1}) \bmod 99\,041 = (N! \bmod 99\,041) \cdot (fr_{x_1}! \cdot fr_{x_2}! \cdot \dots \cdot fr_{x_p}!)^{-1} \bmod 99\,041$, unde $(fr_{x_1}! \cdot fr_{x_2}! \cdot \dots \cdot fr_{x_p}!) \cdot (fr_{x_1}! \cdot fr_{x_2}! \cdot \dots \cdot fr_{x_p}!)^{-1} \equiv 1 \bmod 99\,041$.

Complexitate timp: $O(N)$.

Subtask Diametrele discurilor nu sunt distincte, fără alte restricții

Folosindu-ne de triunghiul lui Pascal, generăm toate combinările C_i^j , $1 \leq i \leq K$ și $1 \leq j < i$. Recurența va fi: $C_i^j = (C_{i-1}^{j-1} + C_{i-1}^j) \bmod 99\,041$. Cazuri de bază: $C_i^i = 1$ și $C_i^0 = 1$. O primă intuiție ar fi să observăm că discurile care au același diametru trebuie obligatoriu puse pe tije diferite, astfel încât să se respecte proprietatea de ordonare strict crescătoare. Numărul de moduri de a pune x discuri de același diametru pe K tije este C_K^x . Dacă încercăm să plasăm mai multe submulțimi de discuri de diametre egale, disjuncte, pe cele K tije, cu $\max_{1 \leq i \leq N} fr_i < K$, vom observa că, în anumite configurații, unele tije rămân goale. Spre exemplu, având șirul diametrelor: 1, 1, 2, 2, 3, 3 și $K = 3$, una dintre configurațiile obținute este: tija 1: 1, 2, 3 și tija 2: 1, 2, 3. Tija 3 rămâne liberă.

Pentru a remedia acest lucru, definim produsul

$$prod_i = (C_i^{fr_{x_1}} \cdot C_i^{fr_{x_2}} \cdot C_i^{fr_{x_3}} \cdot \dots \cdot C_i^{fr_{x_p}}) \cdot C_K^{K-i}$$

Acesta reprezintă numărul de moduri de a așeza discurile pe cel mult i tije și de a păstra $K - i$ tije libere. Pentru un i arbitrar, există C_K^{K-i} moduri de a fixa cele $K - i$ tije libere. Având acest produs calculat, ne putem folosi de principiul includerii și excluderii, răspunsul nostru fiind egal cu $\sum_{i=K}^1 prod_i \cdot (-1)^{K-i} \bmod 99\,041$.

Luăm exemplul de mai sus.

Sunt 3 moduri de a așeza fiecare pereche de discuri de diametru egal, așadar $3^3 = 27$ moduri de a așeza discurile pe maximum 3 tije.

Pentru 2 tije fixate, există un mod unic de a așeza discurile astfel încât să nu existe 2 discuri de același diametru pe aceeași tijă. Există un singur mod deoarece $fr_{x_1} = fr_{x_2} = fr_{x_3} = 2$. Astfel, numărul de moduri de a așeza discurile pe maximum 2 tije este $1 \cdot C_3^2$ (numărul de moduri de a fixa cele 2 tije) $= 1 \cdot C_3^1$ (numărul de moduri de a permuta tija liberă rămasă).

Pentru cazul cu o tijă nu există soluție, deoarece $\max_{1 \leq i \leq N} fr_i = 2 > 1$.

Răspunsul final, în acest caz, va fi: $27 - 3 = 24$.

Complexitate timp: $O(N \cdot K)$.

Soluție instr. Cătălin Frâncu – programare dinamică

Cazul cu diametre distincte

Să considerăm mai întâi cazul în care diametrele discurilor sunt distincte. Fie $D_{i,j}$ numărul de moduri de a așeza cele mai mari i discuri pe exact j tije, definit pentru $1 \leq i \leq N$ și $1 \leq j \leq \min(i, K)$. Cazul de bază al recurenței este $D_{0,0} = 1$. Putem defini recurent $D_{i,j}$ ținând cont de două cazuri posibile și independente:

1. foloseam deja j tije după primele $i - 1$ discuri, caz în care putem așeza discul al i -lea pe oricare dintre aceste j tije, sau
2. foloseam $j - 1$ tije după primele $i - 1$ discuri, caz în care trebuie să ocupăm o tijă nouă și putem așeza discul al i -lea pe oricare dintre cele $K - (j - 1)$ tije libere.

Precizăm pentru completitudine că $D_{i,j}$ nu poate proveni din nicio valoare $D_{i-1,j'}$ unde $j' < j - 1$ (deoarece ar trebui ca discul al i -lea să ocupe două sau mai multe tije simultan) sau $j' > j$ (deoarece nu putem elibera tije). Rezultă formula:

$$D_{i,j} = j \cdot D_{i-1,j} + (K - j + 1) \cdot D_{i-1,j-1}$$

Nu este necesar pentru obținerea punctelor pe subtask-uri, dar pentru un plus de eficiență să observăm că linia D_i depinde doar de linia D_{i-1} , deci putem implementa formula folosind doar doi vectori de mărime K .

Complexitatea soluției este $O(KN)$, deoarece dimensiunea matricei D este $N \times K$ și calculăm fiecare termen în $O(1)$.

Cazul cu diametre egale

Acum să generalizăm aceste observații pentru cazul în care diametrele discurilor pot fi egale. Din datele de intrare vom reține doar frecvența fiecărui diametru i , notată cu fr_i ca mai sus. Vom procesa diametrele în ordine descrescătoare. Redefinim $D_{i,j}$ ca numărul de moduri de a așeza discurile cu diametrele $N, N - 1, \dots, i$ pe exact j tije. Cazul de bază al recurenței este $D_{N+1,0} = 1$.

Formula de recurență este acum mai complexă. Un grup de fr_i discuri identice va ocupa un număr, să-l numim p , de tije deja ocupate la pasul $i + 1$ și $fr_i - p$ tije libere. Pentru un p fixat, dacă la pasul i dorim exact j tije ocupate, atunci la pasul $i + 1$ trebuia să avem $j - fr_i + p$ tije ocupate și, prin diferență, $K - j + fr_i - p$ tije libere. Ținem cont și de faptul că orice combinație de p tije dintre cele $j - fr_i + p$ ocupate și orice combinație de $fr_i - p$ tije dintre cele $K - j + fr_i - p$ libere sunt permise și independente. Astfel obținem formula:

$$D_{i,j} = \sum_{p=0}^{fr_i} D_{i+1,j-fr_i+p} \cdot C_{j-fr_i+p}^p \cdot C_{K-j+fr_i-p}^{fr_i-p}$$

Este bine să ne gândim la inegalitățile dintre aceste valori, pentru ca parametrii combinațiilor și ai matricei D să fie corecți. Din fericire, valorile se aliniază foarte frumos:

1. $0 \leq p \leq fr_i$: Evident din definiția lui p .
2. $fr_i \leq j$: Enunțul impune ca discurile identice să stea pe tije separate.
3. $j \leq K$: Evident, există doar K tije.
4. De aici rezultă și $(K - j) + (fr_i - p) \geq 0$

Implementarea este, pur și simplu:

```

d[n + 1][0] = 1;
for (int i = n; i >= 1; i--) {
    for (int j = f[i]; j <= k; j++) {
        long long sum = 0;
        for (int p = 0; p <= f[i]; p++) {
            sum += (long long)d[i + 1][j - f[i] + p] *
                comb[j - f[i] + p][p] *
                comb[k - j + f[i] - p][f[i] - p];
        }
        d[i][j] = sum % MOD;
    }
}

```

Care este complexitatea? La prima vedere, datorită celor trei `for`-uri, și deoarece $fr_i \leq n$, obținem $O(KN^2)$. În realitate, putem pune condiția mai strictă $\sum_i fr_i = N$. Concret, pentru un i fixat efortul este:

$$O((K - fr_i + 1) \cdot (fr_i + 1)) \leq O((K + 1) \cdot (fr_i + 1))$$

Însumând pentru toate i -urile, obținem:

$$\sum_{i=1}^N O((K + 1) \cdot (fr_i + 1)) = O(K) \cdot O\left(\sum_{i=1}^N (fr_i + 1)\right) = O(KN)$$

Problema 2. Iarmaroc

Propusă de: stud. Rareș-Andrei Cotoi, Facultatea de Matematică și Informatică, Universitatea „Babeș-Bolyai” Cluj-Napoca

Observații

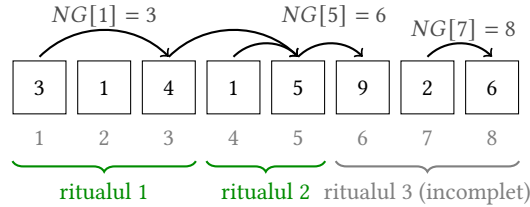
Imediat după finalizarea unui ritual (sau la începutul plimbării), primul produs întâlnit de vizitator îl impresionează întotdeauna, deci fiecare ritual nou pornește exact de la prima tarabă disponibilă după cel precedent. Astfel, pentru fiecare poziție i , definim funcția $NG[i]$ ca fiind prima poziție $j > i$ astfel încât $a_j > a_i$ (*next greater*); dacă nu există, $NG[i] = N + 1$.

La fiecare pas dintr-un ritual, vizitatorul caută primul produs strict mai atractiv decât ultimul (valoarea NG). Astfel, un ritual care pornește de la poziția s selectează pozițiile: $s, NG[s], NG^2[s], \dots, NG^{K-1}[s]$, unde NG^m înseamnă aplicarea funcției NG de m ori. Notăm:

- $comp[i] = NG^{K-1}[i]$: poziția pe care se completează ritualul pornit din i ;
- $f[i] = comp[i] + 1$: poziția de start a ritualului următor.

Astfel, pentru fiecare interogare $[l, r]$, problema se reduce la a afla câte aplicări succesive ale lui f pornind din l se pot efectua astfel încât poziția de completare să rămână $\leq r$.

De exemplu, pentru $a = [3, 1, 4, 1, 5, 9, 2, 6]$, $K = 2$, $l = 1$ și $r = 8$: pornind din $l = 1$, ritualurile complete sunt $\{1, 3\}$ și $\{4, 5\}$; ritualul următor pornește din 6, dar $NG[6] = 9 > r$, deci rămâne incomplet. Răspunsul pentru $[1, 8]$ este 2.



Subtask-urile 1 și 2 ($N, Q \leq 5\,000$)

Pentru acest subtask, putem utiliza o abordare de tip simulare. Pentru fiecare interogare, parcurgem șirul de la l la r , menținând pragul curent s și contorul c de produse cumpărate în ritualul curent. Când $c = K$, incrementăm răspunsul și resetăm $s = 0$, $c = 0$.

```

int s = 0, c = 0, satisfactie = 0;
for (int i = l; i <= r; i++)
    if (a[i] > s) {
        c++; s = a[i];
        if (c == k) { satisfactie++; s = 0; c = 0; }
    }

```

Complexitatea este $O(N \cdot Q)$ și această soluție obține între 20 și 29 de puncte.

Subtask-ul 3 ($l_j = 1$)

Când toate interogările pornesc din $l = 1$, simulăm o singură dată procesul de la 1 la N și reținem într-un vector *final* pozițiile de completare ale fiecărui ritual. Cum fiecare ritual conține cel puțin un element, *final* este strict crescător. Răspunsul pentru interogarea $(1, r)$ este cel mai mare indice m cu $final[m] \leq r$, determinat printr-o căutare binară. Împreună cu simularea, această abordare obține între 35 și 46 de puncte (în funcție de optimizări). Complexitatea este $O(N + Q \log N)$.

Soluție în $O((N + Q)\sqrt{N})$

Soluția completă necesită o accelerare a simulării. O soluție parțială este să precalculăm blocuri de B ritualuri, unde B este o constantă aleasă, apropiată de $\sqrt{N}$. Pentru fiecare poziție de pornire

i precalculăm poziția unde se termină al B -lea ritual. Facem precalcularea naiv, pornind de la poziția i și urmând $f^B[i]$. Apoi, pentru interogarea $[l, r]$:

- Pornim cu $i = l$ și avansăm bloc cu bloc, $i \leftarrow f^B[i]$, cât timp se poate.
- Apoi avansăm ritual cu ritual, $i \leftarrow f[i]$ cât timp se poate.

Complexitatea este $O(N \cdot B)$ pentru precalculare și $O(Q \cdot (N/B))$ pentru interogări. Esența codului este:

```
void build_pointers() {
    for (int i = n; i >= 1; i--) {
        while (ss && (a[i].val >= a[st[ss - 1]].val)) {
            ss--;
        }
        st[ss++] = i;
        if (ss >= k) {
            a[i].end_rit = st[ss - k];
            int num_rits = 0, pos = i;
            while ((num_rits < BLOCK) && a[pos].end_rit) {
                pos = a[pos].end_rit + 1;
                num_rits++;
            }
            if (num_rits == BLOCK) {
                a[i].end_block = pos - 1;
            }
        }
    }
}

int query(int l, int r) {
    int num_rits = 0;
    while (a[l].end_block && (a[l].end_block <= r)) {
        num_rits += BLOCK;
        l = a[l].end_block + 1;
    }
    while (a[l].end_rit && (a[l].end_rit <= r)) {
        num_rits++;
        l = a[l].end_rit + 1;
    }
    return num_rits;
}
```

Soluție în $O((N + Q) \log N)$

Pentru calculul valorilor NG , parcurgem vectorul de la dreapta la stânga, menținând o stivă descrescătoare. La poziția i , eliminăm din stivă toate elementele cu valori $\leq a_i$ (acestea nu mai pot fi *next greater* pentru nicio poziție din stânga lui i), iar $NG[i]$ este vârful stivei rămase (sau $N + 1$ dacă stiva este goală). Apoi îl adăugăm pe i în stivă. Fiecare element este adăugat / scos din stivă cel mult o dată, deci complexitatea timp este $O(N)$.

Cum K poate fi de ordinul lui N , calculul brut al valorilor din vectorul $comp$ ar avea complexitatea $O(N \cdot K)$. Folosim tehnica *binary lifting* peste NG : precalculăm $NG^{2^j}[i]$ pentru toți i și $0 \leq j < \lceil \log_2 N \rceil$, prin recurența

$$NG^{2^0}[i] = NG[i], NG^{2^j}[i] = NG^{2^{j-1}}(NG^{2^{j-1}}[i])$$

Pentru a obține $comp[i]$, descompunem $K - 1$ în baza 2 și compunem nivelurile corespunzătoare (analog exponențierii rapide, dar pe funcții). Construcția are complexitatea $O(N \log N)$.

Aplicăm aceeași tehnică pe funcția $f(f[i] = comp[i] + 1)$:

$$f^{2^0}[i] = f[i], f^{2^j}[i] = f^{2^{j-1}}(f^{2^{j-1}}[i])$$

În plus, reținem $re^{2^j}[i]$ — poziția de completare a *ultimului* ritual dintr-un lanț de 2^j ritualuri consecutive pornind din i : $re^{2^j}[i] = re^{2^{j-1}}(f^{2^{j-1}}[i])$. Cum $NG[i] > i$ pentru orice i , pozițiile de completare ale ritualurilor succesive sunt strict crescătoare. Astfel, dacă ultimul ritual din lanț se completează înainte de r , atunci toate cele precedente se completează.

Pentru o interogare $[l, r]$, pornim din $pos = l$ și parcurgem nivelurile j de la cel mai mare la cel mai mic. Dacă $re^{2^j}[pos] \leq r$, putem efectua 2^j ritualuri complete fără a depăși intervalul: adunăm 2^j la răspuns și avansăm $pos \leftarrow f^{2^j}[pos]$ (metoda este identică cu cea folosită pentru aflarea LCA-ului în arbori). Complexitatea totală este $O(N \log N + Q \log N)$ și soluția aceasta obține 100 de puncte.

Ca optimizare (necesară pentru punctaj maxim), putem implementa *binary lifting* și cu doar două valori pe nod, cu memorie totală $O(N)$ în loc de $O(N \log N)$.

Soluție în $O((N + Q) \cdot \alpha(N))$

În loc să folosim *binary lifting* peste funcția NG (care necesită $O(N \log N)$ timp), observăm că relația tată-fiu definită de $NG[i]$ (primul element mai mare la dreapta) formează o structură de arbore (dacă luăm în calcul nodul fictiv $N + 1$).

Prin efectuarea unei singure parcurgeri în adâncime (DFS) în acest arbore și menținerea căii curente într-o stivă, valoarea $comp[i] = NG^{K-1}[i]$ se determină în $O(1)$ pentru fiecare nod, fiind al $(K - 1)$ -lea strămoș de pe calea curentă. Această metodă reduce precalcularea la $O(N)$.

Pentru a elimina factorul logaritm per interogare, vom procesa toate cerințele $[l_j, r_j]$ *offline*, grupându-le sau sortându-le crescător după capătul din dreapta r_j .

Pe măsură ce un indice dr avansează de la 1 la N , simulăm parcurgerea iarमारocului. Un ritual care începe la o poziție i devine „completat” (sau *activat*) în fereastra curentă doar în momentul în care indicele atinge exact poziția sa de final, adică $dr = comp[i]$.

Pentru a susține această parcurgere de tip *sweep* (baleiere), avem nevoie de o structură de date care să ne permită executarea eficientă a două tipuri de operații:

- **Update:** Când $dr = comp[i]$, trebuie să marcăm ritualul curent ca fiind complet și să creăm o legătură de la poziția de start i către poziția de unde ar începe următorul ritual, adică $f[i]$.
- **Query:** Când dr ajunge la capătul din dreapta al unei interogări ($dr = r_j$), trebuie să aflăm câte astfel de legături consecutive (ritualuri complete) pot fi parcurse pornind de la poziția de start l_j .

Practic, noi dorim să profităm de observația că doi vizitatori care își încheie un ritual la aceeași poziție dr vor avea, începând cu poziția $dr + 1$, un comportament identic și vor completa același număr de ritualuri. De aceea, ne dorim o structură de date care să calculeze numerele de ritualuri pentru acești vizitatori înainte și după joncțiune. Desigur, la model general, fiecare vizitator poate trece prin mai multe astfel de joncțiuni, „revărsându-se” în grupuri tot mai mari de vizitatori.

O structură eficientă care rezolvă aceste tipuri de operații este **DSU**.

- $parent[x]$ – poziția de start a următorului ritual neactivat după poziția x .
- $cnt[x]$ – numărul de ritualuri complete comprimate pe muchia dintre x și $parent[x]$.

La momentul activării unui ritual i (atunci când dr a ajuns la $comp[i]$), îl legăm de următorul ritual disponibil setând $parent[i] = f[i]$ și $cnt[i] = 1$. Inițial, această legătură reprezintă doar faptul că am completat un singur ritual și știm unde începe următorul.

Partea interesantă are loc în momentul interogării. Pentru a afla câte ritualuri complete putem efectua pornind de la un l_j , apelăm funcția $find(l_j)$. Această funcție nu doar că parcurge legăturile pentru a găsi „capătul de linie” (un ritual neactivat încă în fereastra curentă), ci execută și o **compresie a căii** la întoarcerea din recursivitate:

- Fiecare nod vizitat pe traseu este deconectat de la tatăl său intermediar și este legat **direct** de rădăcina lanțului.
- În același timp, contorul cnt al fiecărui nod este actualizat prin însumarea tuturor contoarelor peste care tocmai am sărit.

Datorită acestui mecanism de acumulare, o valoare cnt care a pornit de la 1 se transformă în suma totală a ritualurilor complete de pe lanț. Astfel, răspunsul final pentru interogare este pur și simplu extras din a doua valoare returnată de apel, adică $find(l_j).second$, operația devenind extrem de rapidă pentru orice apeluri viitoare pe aceleași rute.

```

pair<int, int> find(int x) {
    if (parent[x] == x) {
        return {x, 0};
    }

    pair<int, int> res = find(parent[x]);
    cnt[x] += res.second;
    parent[x] = res.first;

    return {parent[x], cnt[x]};
}

```

Analiza Complexității

Pentru a înțelege de unde provine complexitatea finală a soluției, vom descompune algoritmul în pașii săi principali:

1. **Precalcularea arborilor:** $O(N)$
2. **Gruparea interogărilor:** $O(N+Q)$. Deoarece capetele intervalelor sunt numere naturale până la N , putem sorta interogările după capătul din dreapta dr folosind un vector de liste. Această etapă necesită $O(N)$ timp pentru inițializarea listelor și $O(Q)$ timp pentru adăugarea celor Q interogări.
3. **Sweeping și operațiile DSU:** $O((N+Q) \cdot \alpha(N))$. Bucla principală parcurge cele N poziții din iarmaroc. Pe parcursul acestei bucle:
 - Efectuăm exact N operații de **Activare** (legarea $parent[i] = f[i]$), corespunzătoare celor N ritualuri posibile.
 - Efectuăm exact Q operații de **Interogare** (apeluri $find(l_j)$), câte una pentru fiecare cerință.

În total, structura DSU execută $N+Q$ operații. Datorită mecanismului de compresie a căii, deoarece legăturile respectă o direcție strictă spre dreapta, costul amortizat al fiecărei operații $find$ va fi $O(\alpha(N))$, unde α este funcția inversă lui Ackermann.

Adunând pașii de mai sus, complexitatea totală de timp este dominată de operațiile DSU, rezultând $O((N+Q)\alpha(N))$. Memoria utilizată este de asemenea foarte eficientă, $O(N+Q)$, pentru stocarea elementelor, a structurii DSU și a listelor de interogări.

Problema 3. Sortare00

Propusă de: prof. Ionel-Vasile Piț-Rada, Colegiul Național Traian, Drobeta Turnu Severin

Subtask-ul $N \leq 5$

Putem face un algoritm tip backtracking sau BFS pentru a ajunge în starea sortată. Ca să ne dăm seama dacă am mai trecut sau nu printr-o stare, avem diverse posibilități. Două dintre ele sunt:

1. Convertim permutarea într-o bază de numerație. Va fi nevoie să transformăm din permutare în număr și invers, în funcție de context.
2. Ținem minte permutarea la nivel de vector. Vom folosi memorie suplimentară, dar va fi mai simplu de lucrat.

Putem folosi și structuri precum `std::unordered_map` pentru a marca stările prin care deja am trecut, în cazul în care ținem starea drept un vector și nu un număr.

Atenție! Este nevoie de reconstituirea drumului. Atunci, pentru fiecare stare, trebuie să reținem mutarea care ne-a adus din starea precedentă.

Algoritm de rezolvare

Ideea constă în a plasa, rând pe rând, câte o valoare pe poziția pe care ar trebui să se afle în șirul sortat. Mai întâi, plasăm valoarea n pe ultima poziție. Apoi, plasăm valoarea $n - 1$ pe penultima poziție, și așa mai departe. Odată ce plasăm o valoare pe poziția ei, nu mai este nevoie să facem mutări acolo.

Să presupunem că vrem să aducem valoarea x pe „ultima” poziție din șir (având deja plasate valorile $x + 1, x + 2, \dots, n$, ultima poziție devine de fapt poziția unde va fi plasat x).

Pe caz general, ne-am dori să ne folosim de 0-uri să facem o serie de mutări astfel încât x să ajungă pe ultima poziție. Exemplu de interschimbare în 3 mutări (x se află pe poziția S în șir, vrem să ajungă pe poziția D; cu Z sunt marcate pozițiile valorilor de 0; cu * sunt marcate valori de pe poziții vecine care nu sunt de interes):

0. Starea inițială:

*	...	*	S	...	Z	Z	...	*	D
---	-----	---	---	-----	---	---	-----	---	---

1. Mutăm valorile de 0 pe pozițiile D - 1 și D (la sfârșit):

*	...	*	S	...	*	D	...	Z	Z
---	-----	---	---	-----	---	---	-----	---	---

2. Mutăm valorile de 0 pe pozițiile S - 1 și S:

*	...	Z	Z	...	*	D	...	*	S
---	-----	---	---	-----	---	---	-----	---	---

3. Mutăm valorile de 0 pe pozițiile lor inițiale:

*	...	*	D	...	Z	Z	...	*	S
---	-----	---	---	-----	---	---	-----	---	---

Observăm că, după 2 pași, x ajunge la sfârșit. Dacă mai facem un pas în plus, putem aduce valorile de 0 la poziția lor inițială. Însă, în funcție de poziția lui x raportat la poziția valorilor de 0, respectiv la poziția unde vrem să îl plasăm, pot apărea mai multe cazuri particulare.

În acest moment avem de luat o decizie: alegem o variantă mai eficientă din punct de vedere al numărului de operații, dar care va trebui să trateze mai multe cazuri particulare, sau o variantă mai puțin eficientă, care are mai puține cazuri particulare de tratat.

Soluția 1 (Ioan-Cristian Pop)

Având în vedere că pe caz general putem face 3 mutări (raportat la 4, câte ne permitem în medie) ca să plasăm o valoare anume pe o poziție anume, încercăm a doua variantă. Vom menține poziția valorilor de 0 la începutul șirului, ca să reducem numărul de cazuri de analizat. Cât timp cazurile particulare nu vor face în medie mai mult de 4 pași, putem continua pe această idee.

Cazuri posibile

Avem următoarele cazuri posibile, pentru o valoare x (aflată pe poziția S, vrem să ajungă pe poziția D):

- **Caz fericit:** x se află pe poziția D. Nu avem nimic de făcut, trecem mai departe.
- **Caz general:** x nu are la stânga sau la dreapta 0, iar x nu este pe poziția D - 1. Vom face o interschimbare în 3 pași, cum am arătat mai sus.
- **Caz particular I:** x se află pe poziția D - 1 ($S = D - 1$). În acest caz, scopul este să separăm S de D ca să nu facă parte din aceeași pereche. Pașii sunt următorii:

0. Starea inițială:

Z	Z	...	*	*	*	S	D
---	---	-----	---	---	---	---	---

1. Mutăm valorile de 0 pe antepenultima și penultima poziție:

*	S	...	*	*	Z	Z	D
---	---	-----	---	---	---	---	---

2. Mutăm valorile de 0 două poziții mai la stânga:

*	S	...	Z	Z	*	*	D
---	---	-----	---	---	---	---	---

3. Mutăm valorile de 0 pe ultimele două poziții:

*	S	...	*	D	*	Z	Z
---	---	-----	---	---	---	---	---

4. Mutăm valorile de 0 pe primele două poziții:

Z	Z	...	*	D	*	*	S
---	---	-----	---	---	---	---	---

Observăm că se păstrează pozițiile valorilor de 0, făcând 4 mutări.

- **Caz particular II: x are la stânga valoarea 0.** În acest caz, scopul este să plasăm o valoare nenulă la stânga lui x ca să-l putem muta. Mutările sunt următoarele:

0. Starea inițială:

Z	Z	S	...	*	D
---	---	---	-----	---	---

1. Mutăm valorile de 0 la final

*	D	S	...	Z	Z
---	---	---	-----	---	---

2. Mutăm valoarea de pe poziția S la final

*	Z	Z	...	D	S
---	---	---	-----	---	---

Observăm că acest algoritm plasează valorile de 0 cu o poziție mai la dreapta. Cazul general și cazul particular I nu vor fi afectate de acest lucru. Dacă intrăm de două ori pe cazul particular II, atunci putem să mai facem o mutare în plus ca să aducem valorile de 0 de pe pozițiile 2 și 3 la pozițiile 0 și 1.

0. Starea inițială:

*	*	Z	Z	...
---	---	---	---	-----

1. Mutăm valorile de 0 la început:

Z	Z	*	*	...
---	---	---	---	-----

- **Caz particular III: valoarea x se află pe prima poziție în șir.** Putem ajunge în acest caz numai dacă am trecut printr-un caz particular II, plasând o valoare pe prima poziție. Pașii sunt următorii:

0. Starea inițială:

S	Z	Z	*	*	...	*	D
---	---	---	---	---	-----	---	---

1. Mutăm valorile de 0 cu 2 poziții mai la dreapta:

S	*	*	Z	Z	...	*	D
---	---	---	---	---	-----	---	---

2. Mutăm acum valorile de 0 pe primele două poziții:

Z	Z	*	S	*	...	*	D
---	---	---	---	---	-----	---	---

5. Aplicăm algoritmul pe caz general (3 pași):

Z	Z	*	D	*	...	*	S
---	---	---	---	---	-----	---	---

Observăm că, pe acest caz, vom face 5 pași. Însă, putem ajunge în acest caz numai dacă am trecut prin cazul particular II, care face 2 pași. Așa că, în medie, vom face câte 3,5 pași pe parcursul a două ture.

Analiză și concluzii

Se observă că, în medie, vom face cel mult 4 pași per tură. Cele mai nefavorabile cazuri sunt cazul particular I și combinația de caz particular II și III.

Când șirul rămas este destul de scurt (cel mult 7: două zerouri și valorile de la 1 la 5), din acest punct vom folosi algoritmul descris pentru primul subtask. Numărul de cazuri particulare devine destul de mare, iar numărul de permutări posibile este scăzut, așa că nu vom face foarte mulți pași (atât din punct de vedere al numărului de mutări, cât și din punct de vedere al complexității de timp și de memorie).

Soluția 2 (Cătălin Frâncu)

Iată o soluție pe același principiu cu Soluția 1, dar care nu insistă să readucă zerourile pe primele două poziții. Ca să ținem sub control numărul de cazuri de tratat, procedăm astfel:

1. Dacă maximul este deja pe ultima poziție, nu avem nimic de făcut.
2. Altfel, mai întâi aducem zerourile pe ultimele două poziții. Aceasta poate necesita zero mutări (dacă sunt deja acolo) sau două mutări (dacă zerourile sunt pe antepenultima și penultima poziție) sau o mutare (în restul cazurilor).
3. Apoi aducem maximul pe ultima poziție. Aceasta necesită o singură mutare în toate cazurile cu excepția cazului în care maximul este pe prima poziție și trebuie mai întâi scos de acolo, caz în care este nevoie de 4 mutări.

Esența codului este:

```
void greedy_last_element() {
    if (a[n - 1] != n - 2) {
        // Adu gaura pe pozițiile n-2 și n-1.
        if (hole_pos == n - 3) {
            make_move(0);
            make_move(n - 2);
        } else if (hole_pos != n - 2) {
            make_move(n - 2);
        }

        // Caută poziția maximului.
        int pos = ...;

        // Mută maximul la coadă.
        if (pos) {
            make_move(pos - 1);
        } else {
            make_move(2);
            make_move(0);
            make_move(n - 2);
            make_move(1);
        }
    }
}
```

```
n--;  
}
```

Analiză

Ocazional pot fi necesari 6 pași pentru o iterație: 2 pentru mutarea zerourilor la coadă, apoi 4 pentru mutarea maximului la coadă. Însă combinația 2+4 va fi rară, deoarece cei 4 pași lasă zerourile pe pozițiile 1 și 2, ceea ce la iterația următoare va necesita un singur pas pentru mutarea acestora la coadă.

Totuși, pentru acest algoritm determinist, $5n$ pași este un rezultat plauzibil pe teste adverse: putem concepe un test care, studiind algoritmul de mai sus, va cauza la fiecare iterație ca maximul să apară pe prima poziție.

Dar soluția are avantajul că poate înlocui mutarea `make_move(2)` cu o mutare la aproape orice altă poziție. Pozițiile 2 și 3 sunt doar spațiu de manevră, dar putem folosi 3 și 4 sau 4 și 5 etc. Nu putem concepe teste adverse care să cauzeze $5n$ pași pentru toate variantele de program. Și mai bine, putem să generăm aleatoriu poziția acelei mutări, caz în care obținem o estimare probabilistică a numărului de mutări (similar, de exemplu, cu algoritmul quicksort cu pivot randomizat). În practică, numărul de mutări este puțin peste $2n$.

Echipa

Problemele pentru această etapă au fost pregătite de:

- prof. Emanuela Cerchez, Colegiul Național „Emil Racoviță” Iași
- prof. Ionel-Vasile Piț-Rada, Colegiul Național Traian, Drobeta Turnu Severin
- șef lucrări dr. Mihai Nan, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- prof. Dorin-Mircea Rotar, Colegiul Național „Samuil Vulcan” Beiuș
- prof. Mihai Bunget, Colegiul Național „Tudor Vladimirescu” Târgu-Jiu
- prof. Ciprian Cheșcă, Liceul Tehnologic Grigore Moisil Buzău
- prof. Gheorghe-Eugen Nodea, Centrul Județean de Excelență Gorj
- prof. Petru Simion Oprea, Liceul Regina Maria Dorohoi
- prof. Zoltan Szabó, Inspectoratul Școlar Județean Mureș
- prof. Marius Nicoli, Colegiul Național Frații Buzești Craiova
- stud. Alin Gabriel Răileanu, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- instr. Cătălin Frâncu, Nerdvana 2.0 București
- stud. Aureliu Valentin Antonie, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- stud. Ioan-Cristian Pop, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București

- prof. Dana Lica, Centrul Județean de Excelență Prahova
- prof. Bogdan-Ioan Popa, Liceul Teoretic Just4Kids București
- stud. Cristian Luchian, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Andrei Boacă, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Rareș-Andrei Cotoi, Facultatea de Matematică și Informatică, Universitatea „Babeș-Bolyai” Cluj-Napoca
- stud. Lucian-Andrei Badea, TU Delft
- prof. Marinel Șerban, Centrul Județean de Excelență Iași